

ANALYSE DU STOCK ET DES VENTES DU SITE BOTTLENECK

✓ OBJECTIF DE CE NOTEBOOK

Bienvenue dans l'outil plébiscité par les analystes de données Jupyter.

Il s'agit d'un outil permettant de mixer et d'alterner code, texte et graphiques.

Cet outil est formidable pour plusieurs raisons:

- Il permet de tester des lignes de codes au fur et à mesure de votre rédaction, de constater immédiatement le résultat d'une instruction, de la corriger si nécessaire.
- Il permet aussi de rédiger du texte pour expliquer l'approche suivie ou les résultats d'une analyse et de le mettre en forme grâce à du code html ou plus simple avec **Markdown**
- Il est possible d'ajouter des graphiques

Pour vous aider dans vos premiers pas à l'usage de Jupyter et de Python, nous avons rédigé ce notebook en vous indiquant les instructions à suivre.

Il vous suffit pour cela de saisir le code Python répondant à l'instruction donnée.

Vous verrez de temps à autre le code Python répondant à une instruction donnée mais cela est fait pour vous aider à comprendre la nature du travail qui vous est demandé.

Et gardez à l'esprit qu'il n'y a pas de solution unique pour résoudre un problème et qu'il y a autant de résolutions de problèmes que de développeurs ;)...

Etape 1 - Importation des librairies et chargement des fichiers

1.1 - Importation des librairies et paramétrage

```
#Importation de la librairie Pandas
import pandas as pd
```

```
#Importation de la librairie NumPy
import numpy as np
```

```
#Importation de la librairie Matplotlib
import matplotlib.pyplot as plt
```

```
#Importation de la librairie Seaborn
import seaborn as sns
```

```
#Afficher toutes les colonnes d'un dataframe et monter le maximum d
pd.set_option('display.max_columns', None)
pd.set_option('display.max_rows', 100)
```

1.2 - Chargements des fichiers

```
#Connecter au Google Drive
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
#Importation du fichier web.xlsx
df_web = pd.read_excel('/content/drive/MyDrive/Colab Notebooks/Data
/usr/local/lib/python3.12/dist-packages/openpyxl/worksheet/_reader.p
warn(msg)
```

```
#Importation du fichier erp.xlsx
df_erp = pd.read_excel('/content/drive/MyDrive/Colab Notebooks/Data
/usr/local/lib/python3.12/dist-packages/openpyxl/worksheet/_reader.p
warn(msg)
```

```
#Importation du fichier liaison.xlsx
df_liaison = pd.read_excel('/content/drive/MyDrive/Colab Notebooks/
/usr/local/lib/python3.12/dist-packages/openpyxl/worksheet/_reader.p
warn(msg)
```

Etape 2 - Analyse exploratoire des fichiers

2.1 - Analyse exploratoire du fichier erp.xlsx

```
#Consulter les dimensions du dataset et les caractéristiques des cc
df_erp.info()
df_erp.describe(include = 'all')
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 825 entries, 0 to 824
Data columns (total 6 columns):
#   Column                Non-Null Count  Dtype
---  -
0   product_id            825 non-null    int64
1   onsale_web            825 non-null    int64
2   price                 825 non-null    float64
3   stock_quantity        825 non-null    int64
4   stock_status          825 non-null    object
5   purchase_price        825 non-null    float64
dtypes: float64(2), int64(3), object(1)
memory usage: 38.8+ KB
```

	product_id	onsale_web	price	stock_quantity	stock_status
count	825.000000	825.000000	825.000000	825.000000	825.000000
unique	NaN	NaN	NaN	NaN	NaN
top	NaN	NaN	NaN	NaN	instock
freq	NaN	NaN	NaN	NaN	75
mean	5162.597576	0.867879	32.187697	21.589091	NaN
std	902.644635	0.338828	26.712077	21.933807	NaN
min	3847.000000	0.000000	-20.000000	-10.000000	NaN
25%	4348.000000	1.000000	14.500000	7.000000	NaN
50%	4907.000000	1.000000	24.300000	18.000000	NaN
75%	5805.000000	1.000000	42.000000	30.000000	NaN
max	7338.000000	1.000000	225.000000	145.000000	NaN

```
#Afficher les 5 premières lignes de la table
df_erp.head()
```

	product_id	onsale_web	price	stock_quantity	stock_status	purch
0	3847	1	24.2	16	instock	
1	3849	1	34.3	10	instock	
2	3850	1	20.8	0	outofstock	
3	4032	1	14.1	26	instock	
4	4039	1	46.0	3	outofstock	

```
#Vérifier si il y a des lignes en doublon dans la colonne product_i
print("Il y a {} doublon(s) dans la colonne product_id.".format(df_
```

```
Il y a 0 doublon(s) dans la colonne product_id.
```

2.1.1 - Vérification du stock

```
#Afficher les valeurs distinctes de la colonne stock_status
df_erp['stock_status'].value_counts()
```

	count
stock_status	
instock	733
outofstock	92

```
dtype: int64
```

```
#Création d'une colonne "stock_status_2" en lien avec la colonne "s
df_erp_stock_analyse = df_erp.copy()
df_erp_stock_analyse['stock_status_2'] = np.where(df_erp_stock_anal
```

```
#Comparer les colonnes "stock_status" et "stock_status_2" pour voir
erp_compare_stock = df_erp_stock_analyse["stock_status"] == df_erp_
erp_compare_stock.value_counts()
```

	count
True	821
False	4

dtype: int64

```
#Identifier les lignes où il y a une erreur sur le "stock_status"
df_erp_error_stock = df_erp_stock_analyse[(df_erp_stock_analyse["st
df_erp_error_stock
```

	product_id	onsale_web	price	stock_quantity	stock_status	purchase_price
4	4039	1	46.0	3	outofstock	
398	4885	1	18.7	0	instock	
449	4973	0	10.0	-10	outofstock	
573	5700	1	44.5	-1	outofstock	

2.1.2 - Vérification prix vente vs prix achat

```
#création de la colonne price - purchase_price
df_erp_price_analyse = df_erp.copy()
df_erp_price_analyse['price_compare'] = df_erp_price_analyse['price
```

```
#Vérifier que les valeurs calculées soient positives
erp_compare_price = df_erp_price_analyse["price_compare"] > 0
erp_compare_price.value_counts()
```

	count
price_compare	
True	818
False	7

dtype: int64

```
#Identifier les lignes où il y a une erreur sur le prix
df_erp_error_price = df_erp_price_analyse[(df_erp_price_analyse["pr
df_erp_error_price
```

	product_id	onsale_web	price	stock_quantity	stock_status	purc
151	4233	0	-20.00	0	outofstock	
210	4355	1	12.65	97	instock	
391	4864	0	8.30	0	outofstock	
469	5017	0	-8.00	0	outofstock	
724	6324	0	92.00	18	instock	
739	6594	0	-9.10	19	instock	
817	7196	0	31.00	55	instock	

2.1.3 - Analyse de la variable ONSALE_WEB

```
#Vérification de la colonne onsale_web et des valeurs qu'elle conti  
df_erp['onsale_web'].value_counts()
```

	count
onsale_web	
1	716
0	109

dtype: int64

2.1.4 - Mise en forme du dataframe corrigé

```
#Corriger la ou les données incohérentes
df_erp_fix = df_erp.copy()
df_erp_fix.drop(df_erp_error_stock.index, inplace=True)
df_erp_fix.drop(df_erp_error_price.index, inplace=True)

#Suppression des colonnes redondantes ou inutiles
df_erp_fix.drop(['stock_status'], axis=1, inplace=True)

#Caractéristiques du nouveau dataframe
df_erp_fix.info()
df_erp_fix.describe(include = 'all')
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
Index: 814 entries, 0 to 824
```

```
Data columns (total 5 columns):
```

#	Column	Non-Null Count	Dtype
0	product_id	814 non-null	int64
1	onsale_web	814 non-null	int64
2	price	814 non-null	float64
3	stock_quantity	814 non-null	int64
4	purchase_price	814 non-null	float64

```
dtypes: float64(2), int64(3)
```

```
memory usage: 38.2 KB
```

	product_id	onsale_web	price	stock_quantity	purchase_price
count	814.000000	814.000000	814.000000	814.000000	814.000000
mean	5160.888206	0.874693	32.344963	21.658477	16.8038
std	901.374111	0.331270	26.625313	21.786029	14.1785
min	3847.000000	0.000000	5.200000	0.000000	2.7400
25%	4339.750000	1.000000	14.625000	7.000000	7.5975
50%	4905.500000	1.000000	24.400000	18.000000	12.7200
75%	5804.750000	1.000000	41.950000	30.000000	22.0100
max	7338.000000	1.000000	225.000000	145.000000	137.8100

```
#Répartition de produits vendus en ligne
df_erp_fix['onsale_web'].value_counts()
```

```

              count
onsale_web
1              712
0              102

dtype: int64
```

2.2 - Analyse exploratoire du fichier web.xlsx

```
#Consulter la dimension du dataset et les caractéristiques des cols
df_web.info()
df_web.describe(include = 'all')
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1513 entries, 0 to 1512
Data columns (total 29 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   sku                                    1428 non-null   object
1   virtual                              1513 non-null   int64
2   downloadable                          1513 non-null   int64
3   rating_count                          1513 non-null   int64
4   average_rating                        1430 non-null   float64
5   total_sales                           1430 non-null   float64
6   tax_status                            716 non-null    object
7   tax_class                             0 non-null      float64
8   post_author                           1430 non-null   float64
9   post_date                             1430 non-null   datetime64[ns]
10  post_date_gmt                         1430 non-null   datetime64[ns]
11  post_content                           0 non-null      float64
12  product_type                           1429 non-null   object
13  post_title                             1430 non-null   object
14  post_excerpt                           716 non-null    object
15  post_status                            1430 non-null   object
16  comment_status                         1430 non-null   object
17  ping_status                            1430 non-null   object
18  post_password                           0 non-null      float64
19  post_name                              1430 non-null   object
20  post_modified                          1430 non-null   datetime64[ns]
21  post_modified_gmt                      1430 non-null   datetime64[ns]
22  post_content_filtered                  0 non-null      float64
23  post_parent                            1430 non-null   float64
24  guid                                   1430 non-null   object
25  menu_order                             1430 non-null   float64
```

```
25 menu_order      1430 non-null    float64
26 post_type        1430 non-null    object
27 post_mime_type    714 non-null     object
28 comment_count     1430 non-null    float64
dtypes: datetime64[ns](4), float64(10), int64(3), object(12)
memory usage: 342.9+ KB
```

	sku	virtual	downloadable	rating_count	average_rating	t
count	1428.0	1513.0	1513.0	1513.0	1430.0	
unique	714.0	NaN	NaN	NaN	NaN	
top	14569.0	NaN	NaN	NaN	NaN	
freq	2.0	NaN	NaN	NaN	NaN	
mean	NaN	0.0	0.0	0.0	0.0	
min	NaN	0.0	0.0	0.0	0.0	
25%	NaN	0.0	0.0	0.0	0.0	
50%	NaN	0.0	0.0	0.0	0.0	
75%	NaN	0.0	0.0	0.0	0.0	
max	NaN	0.0	0.0	0.0	0.0	
std	NaN	0.0	0.0	0.0	0.0	

```
#Afficher les premières lignes du dataframe  
df_web.head()
```

	sku	virtual	downloadable	rating_count	average_rating	total_s
0	11862	0	0	0	0.0	
1	16057	0	0	0	0.0	
2	14692	0	0	0	0.0	
3	16295	0	0	0	0.0	
4	15328	0	0	0	0.0	

```
#Observation de la colonne 'post_type'  
df_web['post_type'].value_counts()
```

	count
post_type	
product	716
attachment	714
dtype: int64	

```
#Identifier les lignes des pièces-jointes créant des doublons de sk
index_web_rows_suppr = df_web.loc[df_web['post_type']=='attachment']
```

```
#Identifier les colonnes à supprimer (vides, redondantes, surinform
index_web_columns_suppr = df_web[['virtual','downloadable','rating_
```

```
#Identifier les lignes sans sku
df_web_no_sku = df_web[df_web["sku"].isnull()]
df_web_no_sku.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 85 entries, 8 to 1457
Data columns (total 29 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   sku                                    0 non-null      object
1   virtual                              85 non-null     int64
2   downloadable                         85 non-null     int64
3   rating_count                         85 non-null     int64
4   average_rating                      2 non-null      float64
5   total_sales                         2 non-null      float64
6   tax_status                          2 non-null      object
7   tax_class                           0 non-null      float64
8   post_author                         2 non-null      float64
9   post_date                          2 non-null      datetime64[ns]
10  post_date_gmt                      2 non-null      datetime64[ns]
11  post_content                       0 non-null      float64
12  product_type                       2 non-null      object
13  post_title                         2 non-null      object
14  post_excerpt                       2 non-null      object
15  post_status                        2 non-null      object
16  comment_status                     2 non-null      object
17  ping_status                       2 non-null      object
18  post_password                      0 non-null      float64
19  post_name                         2 non-null      object
20  post_modified                     2 non-null      datetime64[ns]
21  post_modified_gmt                 2 non-null      datetime64[ns]
22  post_content_filtered             0 non-null      float64
23  post_parent                       2 non-null      float64
24  guid                             2 non-null      object
25  menu_order                       2 non-null      float64
26  post_type                        2 non-null      object
27  post_mime_type                   0 non-null      object
28  comment_count                    2 non-null      float64
dtypes: datetime64[ns](4), float64(10), int64(3), object(12)
memory usage: 19.9+ KB
```

```
#Supprimer les colonnes et les lignes identifiées précédemment
df_web_analyse = df_web.copy()
df_web_analyse.drop(index_web_rows_suppr, axis=0, inplace=True)
df_web_analyse.drop(index_web_columns_suppr, axis=1, inplace=True)
df_web_analyse.drop(df_web_no_sku.index, inplace=True)
```

```
#Caractéristiques du dataframe modifié
df_web_analyse.info()
df_web_analyse.describe(include='all')
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 714 entries, 2 to 1509
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  -
0    sku             714 non-null    object
1    total_sales     714 non-null    float64
2    product_type    713 non-null    object
3    post_title      714 non-null    object
dtypes: float64(1), object(3)
memory usage: 27.9+ KB
```

	sku	total_sales	product_type	post_title
count	714.0	714.000000	713	714
unique	714.0	NaN	6	711
top	15662.0	NaN	Vin	Marc Colin Et Fils Chassagne-Montrachet Blanc ...
freq	1.0	NaN	658	2
mean	NaN	8.054622	NaN	NaN
std	NaN	4.161344	NaN	NaN
min	NaN	0.000000	NaN	NaN
25%	NaN	5.000000	NaN	NaN
50%	NaN	8.000000	NaN	NaN
75%	NaN	11.000000	NaN	NaN
max	NaN	36.000000	NaN	NaN

```
#Vérification des 'sku' en doublon
print("Il y a {} doublon(s) dans la colonne sku.".format(df_web_ana
```

```
Il y a 0 doublon(s) dans la colonne sku.
```

```
#Valeurs de la colonne 'sku' ne respectant pas la numérotation généré
web_sku_cleaned = df_web_analyse["sku"].fillna('')
web_numeric = web_sku_cleaned.str.isnumeric()
web_numeric = web_numeric.astype(bool)
web_no_numeric = df_web_analyse[~web_numeric].dropna(subset=['sku'])
web_no_numeric
```

	sku	total_sales	product_type	post_title
272	13127-1	4.0	Vin	Clos du Mont-Olivet Châteauneuf-du-Pape 2007

```
#Suppression du 'bon-cadeau-25-euros' source d'erreur car il sera u
df_web_analyse.drop(df_web_analyse[df_web_analyse['sku'] == 'bon-ca
```

```
#Analyse du 'sku' '13127-1' et cohérence avec le reste
df_web_analyse[(df_web_analyse ["sku"] == 13127) | (df_web_analyse[
```

	sku	total_sales	product_type	post_title
123	13127	5.0	Vin	Clos du Mont-Olivet Châteauneuf- du-Pape 2012
272	13127-	4.0	Vin	Clos du Mont-Olivet Châteauneuf-

```
#Uniformisation du type de 'sku' pour pouvoir trier, grouper...
df_web_analyse['sku'] = df_web_analyse['sku'].astype(str)
df_web_analyse.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 713 entries, 2 to 1509
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  -
0    sku              713 non-null    object
1    total_sales      713 non-null    float64
2    product_type     713 non-null    object
3    post_title       713 non-null    object
dtypes: float64(1), object(3)
memory usage: 27.9+ KB
```

2.3 - Analyse exploratoire du fichier liaison.xlsx

```
#Consulter la taille du dataframe et les caractéristiques des colon
df_liaison.info()
df_liaison.describe(include = 'all')
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 825 entries, 0 to 824
Data columns (total 2 columns):
#   Column          Non-Null Count  Dtype
---  -
0   id_web           734 non-null    object
1   product_id       825 non-null    int64
dtypes: int64(1), object(1)
memory usage: 13.0+ KB
```

	id_web	product_id
count	734.0	825.000000
unique	734.0	NaN
top	16230.0	NaN
freq	1.0	NaN
mean	NaN	5162.597576
std	NaN	902.644635
min	NaN	3847.000000
25%	NaN	4348.000000
50%	NaN	4907.000000
75%	NaN	5805.000000
max	NaN	7338.000000

```
#Vérifier le caractère unique de "product_id"
print("Il y a {} doublon(s) dans la colonne product_id.".format(df_
```

```
Il y a 0 doublon(s) dans la colonne product_id.
```

```
#Vérifier s'il y a des articles sans correspondance
erp_verif_corresp = df_liaison['id_web'].notnull()
erp_verif_corresp.value_counts()
```

```

      count
id_web
True      734
False      91

```

```
dtype: int64
```

```
#Identifier les lignes 'id_web' vides
df_liaison_no_id_web = df_liaison[df_liaison["id_web"].isnull()]
df_liaison_no_id_web.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 91 entries, 19 to 821
Data columns (total 2 columns):
#   Column          Non-Null Count  Dtype
---  -
0   id_web           0 non-null     object
1   product_id       91 non-null    int64
dtypes: int64(1), object(1)
memory usage: 2.1+ KB
```

```
#Supprimer les lignes correspondant à des id_web vides
df_liaison_analyse = df_liaison.copy()
df_liaison_analyse.drop(df_liaison_no_id_web.index, axis=0, inplace=True)
df_liaison_analyse['id_web'] = df_liaison_analyse['id_web'].astype(object)
df_liaison_analyse.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 734 entries, 0 to 824
Data columns (total 2 columns):
#   Column          Non-Null Count  Dtype
---  -
0   id_web           734 non-null    object
1   product_id       734 non-null    int64
dtypes: int64(1), object(1)
memory usage: 17.2+ KB
```

Etape 3 - Jonction des fichiers

Etape 3.1 - Jonction du fichier df_erp et df_liaison

```
#Fusion des fichiers df_erp et df_liaison
df_merge = pd.merge(df_erp_fix, df_liaison_analyse, on='product_id'
df_merge.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 814 entries, 0 to 813
Data columns (total 6 columns):
#   Column                Non-Null Count  Dtype
---  -
0   product_id            814 non-null    int64
1   onsale_web            814 non-null    int64
2   price                 814 non-null    float64
3   stock_quantity        814 non-null    int64
4   purchase_price        814 non-null    float64
5   id_web                729 non-null    object
dtypes: float64(2), int64(3), object(1)
memory usage: 38.3+ KB
```

```
#Identification des 'product_id' présent dans la table de liaison n
df_full_merge = pd.merge(df_erp_fix, df_liaison_analyse, on='produc
df_erreurs = df_full_merge[df_full_merge['_merge'].isin(['right_onl
df_erreurs
```

	product_id	onsale_web	price	stock_quantity	purchase_price	id
4	4039	NaN	NaN	NaN	NaN	
209	4355	NaN	NaN	NaN	NaN	
390	4864	NaN	NaN	NaN	NaN	
397	4885	NaN	NaN	NaN	NaN	
570	5700	NaN	NaN	NaN	NaN	

```
#Suppression des lignes identifiées
df_merge.drop(df_erreurs.index, axis=0, inplace=True)
df_merge.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 809 entries, 0 to 813
Data columns (total 6 columns):
#   Column                Non-Null Count  Dtype
---  -
0   product_id            809 non-null    int64
1   onsale_web            809 non-null    int64
2   price                 809 non-null    float64
3   stock_quantity       809 non-null    int64
4   purchase_price       809 non-null    float64
5   id_web               724 non-null    object
dtypes: float64(2), int64(3), object(1)
memory usage: 44.2+ KB
```

Etape 3.2 - Jonction du fichier df_merge et df_web

```
#Fusionner les datasets df_merge et df_web
df_merge_final = pd.merge(df_merge, df_web_analyse, left_on='id_web
df_merge_final.reset_index(drop=True)
df_merge_final.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 809 entries, 0 to 808
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   product_id            809 non-null    int64
1   onsale_web            809 non-null    int64
2   price                 809 non-null    float64
3   stock_quantity       809 non-null    int64
4   purchase_price       809 non-null    float64
5   id_web               724 non-null    object
6   sku                  705 non-null    object
7   total_sales          705 non-null    float64
8   product_type         705 non-null    object
9   post_title           705 non-null    object
dtypes: float64(3), int64(3), object(4)
memory usage: 63.3+ KB
```

```
#Vérification si tous les produits présents sur l'extraction des ve  
df_merge_final.loc[(df_merge_final['sku'].notna()) & (df_merge_fina
```

product_id	onsale_web	price	stock_quantity	purchase_price	ic
-------------------	-------------------	--------------	-----------------------	-----------------------	-----------

```
#Correction des lignes incohérentes  
df_merge_final.loc[df_merge_final['product_id'] == 4200, 'onsale_we  
df_merge_final.loc[df_merge_final['product_id'] == 4200]
```

product_id	onsale_web	price	stock_quantity	purchase_price	ic
-------------------	-------------------	--------------	-----------------------	-----------------------	-----------

```
#Vérification de la présence du produit bon-cadeau-25-euros dans le  
df_merge_bon_cadeau=df_merge_final.loc[df_merge_final['id_web'] ==  
df_merge_bon_cadeau
```

product_id	onsale_web	price	stock_quantity	purchase_price	ic
-------------------	-------------------	--------------	-----------------------	-----------------------	-----------

```
#Identification des produits inactifs (0 stock et 0 vente)  
df_product_inactive=df_merge_final.loc[(df_merge_final['stock_quant
```

```
#Affichage des premières lignes de produits inactifs  
df_product_inactive.head(20)
```

	product_id	onsale_web	price	stock_quantity	purchase_price	ic
2	3850	1	20.8	0	10.64	
6	4043	1	60.0	0	29.45	
9	4047	1	18.3	0	9.93	
13	4051	1	7.7	0	4.14	
14	4052	1	33.7	0	18.11	
17	4055	0	86.1	0	37.88	
26	4065	1	19.5	0	9.67	
40	4079	1	37.0	0	19.50	

```
#Nettoyage du dataframe  
df_merge_final.drop(df_product_inactive.index, axis=0, inplace=True)  
df_merge_final.drop(df_merge_bon_cadeau.index, axis=0, inplace=True)  
df_merge_final.drop(['sku', 'onsale_web'], axis=1, inplace=True)
```

```
#Caractéristiques du dataframe
df_merge_final.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 745 entries, 0 to 808
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  -
0   product_id            745 non-null    int64
1   price                 745 non-null    float64
2   stock_quantity        745 non-null    int64
3   purchase_price        745 non-null    float64
4   id_web                686 non-null    object
5   total_sales           684 non-null    float64
6   product_type          684 non-null    object
7   post_title            684 non-null    object
dtypes: float64(3), int64(2), object(3)
memory usage: 52.4+ KB
```

```
#Affichage premières lignes du dataframe
df_merge_final.head()
```

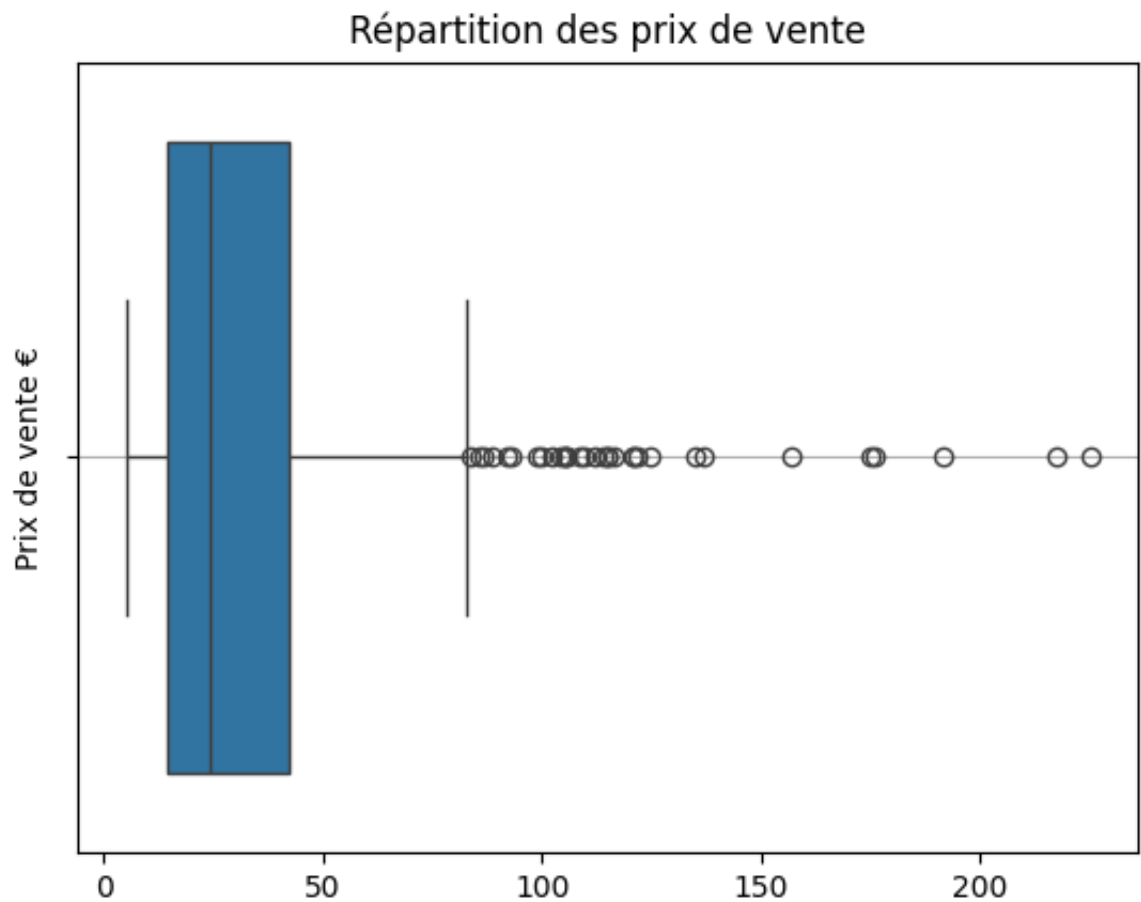
	product_id	price	stock_quantity	purchase_price	id_web	total_s
0	3847	24.2	16	12.88	15298	
1	3849	34.3	10	17.54	15296	

Etape 4 - Analyse univariée des prix

Etape 4.1 - Exploration par la visualisation de données

Etape 4.1.1 - Répartition des prix de vente

```
#Répartition des prix (boîte à moustache)
sns.boxplot(data=df_merge_final ,x='price')
plt.grid(axis='y')
plt.title('Répartition des prix de vente')
plt.ylabel('Prix de vente €')
plt.xlabel('')
plt.show()
```



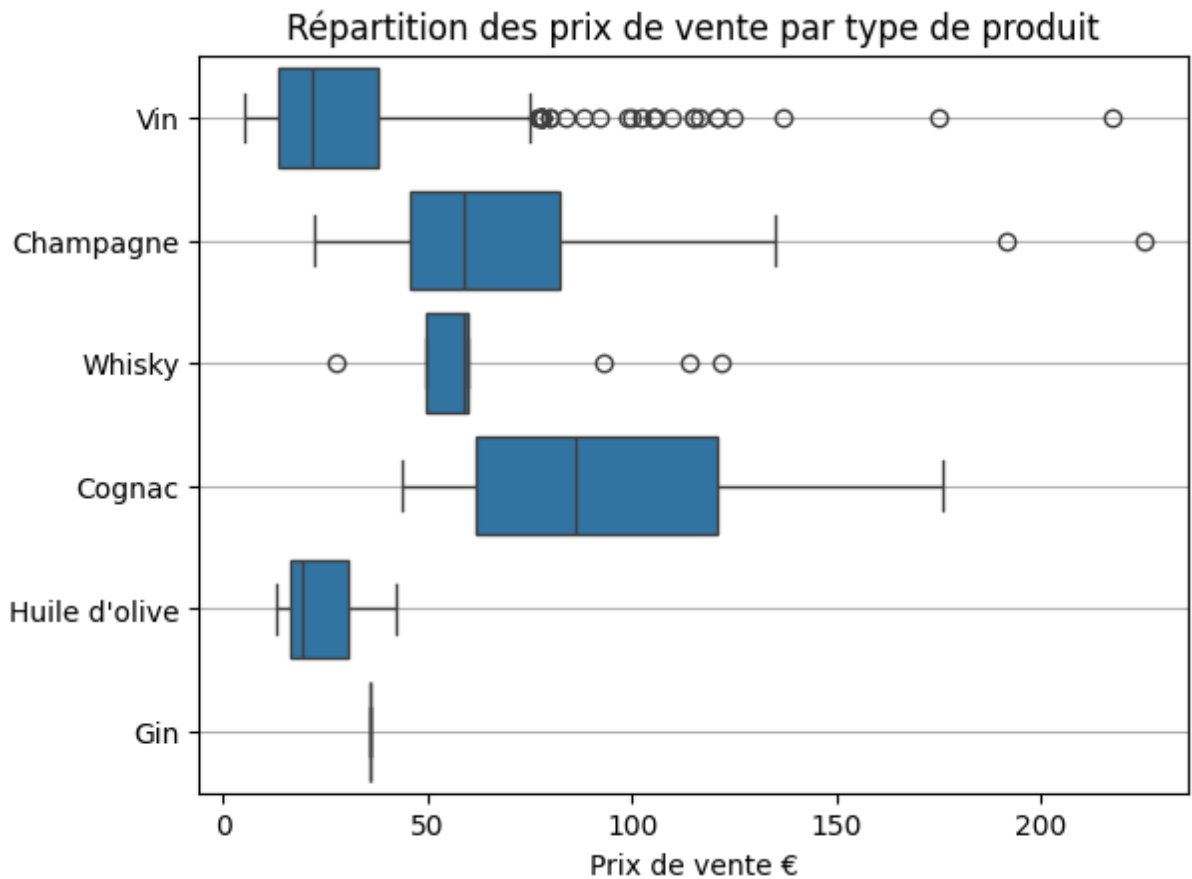
```
#Mesures de dispersion de 'price'  
df_merge_final['price'].describe()
```

	price
count	745.000000
mean	32.444430
std	26.948579
min	5.200000
25%	14.500000
50%	24.400000
75%	42.100000
max	225.000000

dtype: float64

Etape 4.1.2 - Répartition des prix de vente par produit

```
#Répartition des prix par produit (boîte à moustache)
sns.boxplot(data=df_merge_final,x='price', y='product_type')
plt.grid(axis='y')
plt.title('Répartition des prix de vente par type de produit')
plt.ylabel('')
plt.xlabel('Prix de vente €')
plt.show()
```



```
#Mesures de dispersion de 'price' par 'product_type'
df_merge_final.groupby('product_type')['price'].describe()
```

	count	mean	std	min	25%	50%	75%	max
product_type								
Champagne	26.0	72.996154	47.645193	22.5	46.00	59.00	82.125	225.0
Cognac	8.0	97.500000	48.153208	44.0	61.85	86.05	120.625	176.0
Gin	2.0	36.000000	0.000000	36.0	36.00	36.00	36.000	36.0
Huile d'olive	3.0	24.933333	15.291937	13.1	16.30	19.50	30.850	42.0
Vin	632.0	29.235443	23.702588	5.2	13.50	21.75	38.100	217.0
Whisky	13.0	65.830769	27.103302	27.5	49.50	59.00	59.800	122.0

Etape 4.2 - Exploration par l'utilisation de méthodes statistiques

Etape 4.2.1 - Identification par le Z-index

```
#Calculer la moyenne du prix
moyenne_prix = df_merge_final['price'].mean()
print('Prix moyen : {} €'.format(round(moyenne_prix,2)))
#Calculer l'écart-type du prix
ecart_type_prix = df_merge_final['price'].std()
print('Ecart-type : {}'.format(round(ecart_type_prix,2)))
#Calculer le Z-score
z_score_prix = (df_merge_final['price'] - moyenne_prix) / ecart_type_prix
print('\nZ-score :\n{}'.format(round(z_score_prix,2)))
```

```
Prix moyen : 32.44 €
Ecart-type : 26.95
```

```
Z-score :
0      -0.31
1       0.07
3      -0.68
4       0.01
5      -0.05
...
804     0.47
805     0.47
806     0.83
807    -0.22
808    -0.60
Name: price, Length: 745, dtype: float64
```

```
#Quel est le seuil prix dont le z-score est supérieur à 3?
seuil_prix_3 = moyenne_prix + (3 * ecart_type_prix)
print(round(seuil_prix_3,2))
```

```
113.29
```

Etape 4.2.2 - Identification par l'intervalle interquartile

```
#Définir un seuil pour les articles "outliers" en prix
IQR= df_merge_final['price'].quantile(0.75) - df_merge_final['price'].quantile(0.25)
seuil_prix_max = df_merge_final['price'].quantile(0.75) + (1.5 * IQR)
seuil_prix_min = df_merge_final['price'].quantile(0.25) - (1.5 * IQR)
print('Prix max :', round(seuil_prix_max,2))
print('Prix min :', round(seuil_prix_min,2))
```

```
Prix max : 83.5
Prix min : -26.9
```

```
#Définir le catalogue "outliers"
df_outliers=df_merge_final.loc[(df_merge_final['price'] > seuil_pri
print('Total outliers :', df_outliers.shape[0])
```

Total outliers : 32

```
#Analyse des produits concernés
df_outliers.groupby('product_type')[['stock_quantity', 'total_sales
```

	stock_quantity	total_sales
product_type		
Champagne	585	44.0
Cognac	54	16.0
Vin	228	65.0
Whisky	25	7.0

```
#Analyse du taux de marge de ces produits
df_outliers['margin_rate'] = (df_outliers['price']-df_outliers['pur
df_outliers.groupby('product_type')['margin_rate'].mean()
```

	margin_rate
product_type	
Champagne	66.234328
Cognac	120.654513
Vin	94.507130
Whisky	124.264805

dtype: float64

```
#Comparaison avec les taux de marge des autres produits
df_no_outliers=df_merge_final.drop(df_outliers.index).copy()

df_no_outliers['margin_rate'] = (df_no_outliers['price']-df_no_outl

marge_champagne_outliers = df_outliers.loc[df_outliers['product_type
marge_champagne_no_outliers = df_no_outliers.loc[df_no_outliers['pr
difference_champagne = round(marge_champagne_outliers - marge_champ

marge_cognac_outliers = df_outliers.loc[df_outliers['product_type']
marge_cognac_no_outliers = df_no_outliers.loc[df_no_outliers['produ
difference_cognac = round(marge_cognac_outliers - marge_cognac_no_c

marge_vin_outliers = df_outliers.loc[df_outliers['product_type'] ==
marge_vin_no_outliers = df_no_outliers.loc[df_no_outliers['product_
difference_vin = round(marge_vin_outliers - marge_vin_no_outliers,

marge_whisky_outliers = df_outliers.loc[df_outliers['product_type']
marge_whisky_no_outliers = df_no_outliers.loc[df_no_outliers['produ
difference_whisky = round(marge_whisky_outliers - marge_whisky_no_c

df_no_outliers.groupby('product_type')['margin_rate'].mean()
```

	margin_rate
product_type	
Champagne	68.111792
Cognac	116.904357
Gin	109.790210
Huile d'olive	60.068162
Vin	93.815010
Whisky	116.226743

dtype: float64

```
#Conclusion sur les outliers
print("Les taux de marge sont comparables : \n\nChampagne : {} \nCc

Les taux de marge sont comparables :

Champagne : -1.88
Cognac : 3.75
Vin : 0.69
Whisky : 8.04

Cela confirme la cohérence des prix des produits outliers avec un pc
```

Etape 5 - Analyse univariée du CA, des quantités vendues, des stocks et de la marge ainsi qu'une analyse multivariée

Etape 5.1 - Analyse des ventes en CA

```
#####
# Calculer le CA du site web #
#####

df_analyse_ca=df_merge_final.copy()

#Créer une colonne calculant le CA par article
df_analyse_ca['ca'] = df_analyse_ca['price'] * df_analyse_ca['total

#Calculer la somme de la colonne "ca_par_article"
ca_oct = df_analyse_ca['ca'].sum()
print('Chiffre d\'affaire du site web en octobre : {} €'.format(rou

Chiffre d'affaire du site web en octobre : 142253.0 €
```

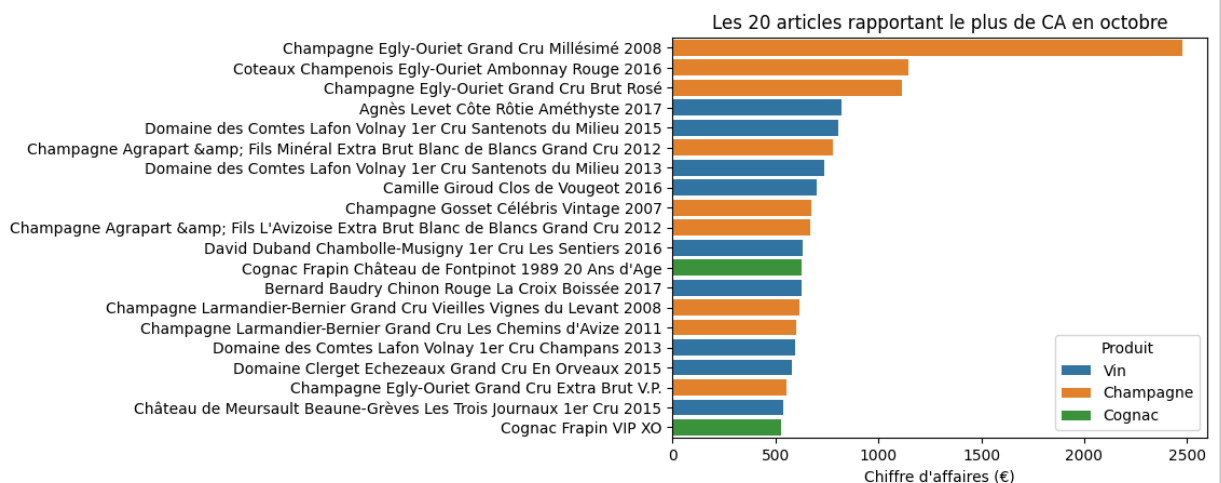
```
#####
# Palmarès des articles en CA #
#####

#Effectuer le tri dans l'ordre décroissant du CA du dataset df_anal
df_analyse_ca.sort_values(by='ca', ascending=False, inplace=True)

#Réinitialiser l'index du dataset par un reset_index
df_analyse_ca.reset_index(drop=True, inplace=True)

#Afficher les 20 premiers articles en CA
df_analyse_ca.head(20)

#Graphique en barre des 20 premiers articles
sns.barplot(data=df_analyse_ca.head(20), x='ca', y='post_title', hu
plt.xlabel('Chiffre d\affaires (€)')
plt.ylabel('')
plt.legend(title='Produit')
plt.title('Les 20 articles rapportant le plus de CA en octobre')
plt.show()
```



```
#####
# Calculer le 20 / 80 en CA #
#####

#Créer une colonne calculant la part du CA de la ligne dans le data
df_analyse_ca['part_ca_total'] = df_analyse_ca['ca'] / df_analyse_c

#Créer une colonne réalisant la somme cumulative de la colonne préc
df_analyse_ca['cumul_part_ca_total'] = df_analyse_ca['part_ca_total

#Identification des produits représentant les 80% en quantités vend
df_analyse_ca['priority_ca'] = np.where(df_analyse_ca['cumul_part_c

#Grâce aux deux colonnes créées précédemment, calculer le nombre d'
df_ca_80=df_analyse_ca.loc[df_analyse_ca['cumul_part_ca_total'] <=

#Afficher la proportion que représente ce groupe d'articles dans le
proportion_ca_pareto=df_ca_80.shape[0] / df_analyse_ca.shape[0]*100
print('{}% des articles représentent 80% du CA en octobre.'.format(

57.58% des articles représentent 80% du CA en octobre.
```

Etape 5.2 - Analyse des ventes en quantité

```
#####
# Palmarès des articles en quantité #
#####

df_analyse_quantity=df_merge_final.copy()

#Effectuer le tri dans l'ordre décroissant de quantités vendues du
df_analyse_quantity.sort_values(by='total_sales', ascending=False,

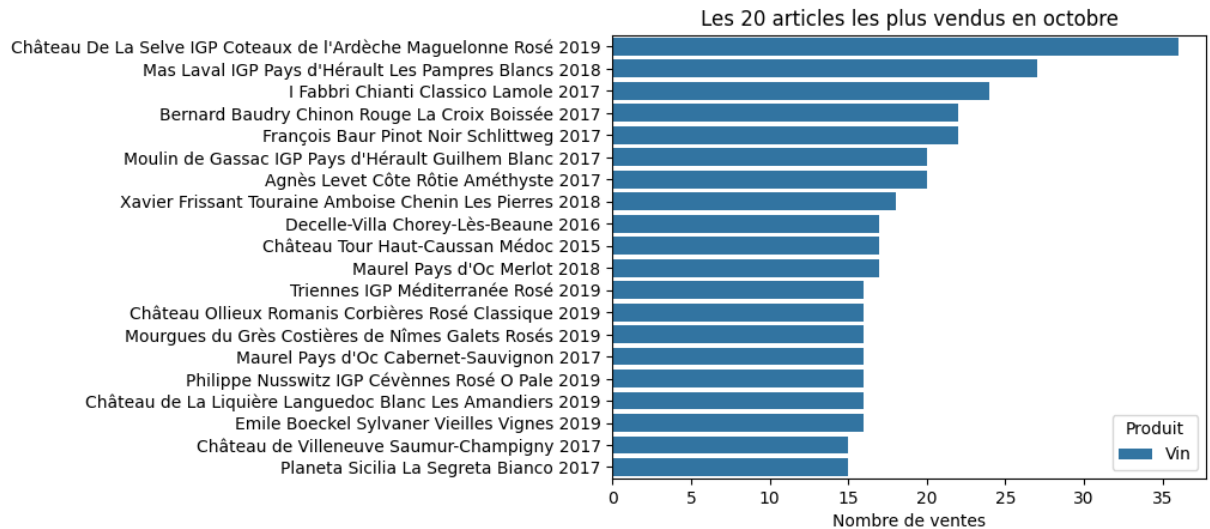
#Réinitialiser l'index du dataset par un reset_index
df_analyse_quantity.reset_index(drop=True, inplace=True)

#Afficher les 20 premiers articles en quantité
df_analyse_quantity.head(20)
```

	product_id	price	stock_quantity	purchase_price	id_web	total_
0	4867	9.9	121	4.86	16148	

1	4203	9.9	74	5.01	15415
2	4275	14.9	62	7.78	14864
3	4647	28.5	45	14.14	16525
4	4726	12.7	0	6.82	14950
5	6129	5.2	68	2.74	14570

```
#Graphique en barre des 20 premiers articles
sns.barplot(data=df_analyse_quantity.head(20), x='total_sales', y='
sns.color_palette(['red', 'green', 'blue', 'orange'])
plt.xlabel('Nombre de ventes')
plt.ylabel('')
plt.legend(title='Produit')
plt.title('Les 20 articles les plus vendus en octobre')
plt.show()
```



```
#####
# Calculer le 20 / 80 en quantité #
#####

#Créer une colonne calculant la part en quantité de la ligne dans l
df_analyse_quantity['part_quantity_total'] = df_analyse_quantity['t

#Créer une colonne réalisant la somme cumulative de la colonne préc
df_analyse_quantity['cumul_part_quantity_total'] = df_analyse_quant

#Identification des produits représentant les 80% en quantités vend
df_analyse_quantity['priority_quantity'] = np.where(df_analyse_quan

#Grâce aux deux colonnes créées précédemment, calculer le nombre d'
df_analyse_quantity_80=df_analyse_quantity.loc[df_analyse_quantity[

#Afficher la proportion que représente ce groupe d'articles dans le
proportion_quantity_pareto=df_analyse_quantity_80.shape[0] / df_ana
print('{}% des articles représentent 80% des quantités vendues en c

57.45% des articles représentent 80% des quantités vendues en octobr
```

Etape 5.3 - Analyse des stocks

```
#####
# Calculer le nombre de mois de stock #
#####
df_analyse_stock=df_merge_final.copy()

#Création de la colonne mois de stock
df_analyse_stock['month_stock'] = df_analyse_stock['stock_quantity'

#Remplacement des "inf" par 0
df_analyse_stock.replace([np.inf, -np.inf], 0, inplace=True)

#Effectuer le tri dans l'ordre décroissant du nombre de mois de stc
df_analyse_stock.sort_values(by='month_stock', ascending=False, inp

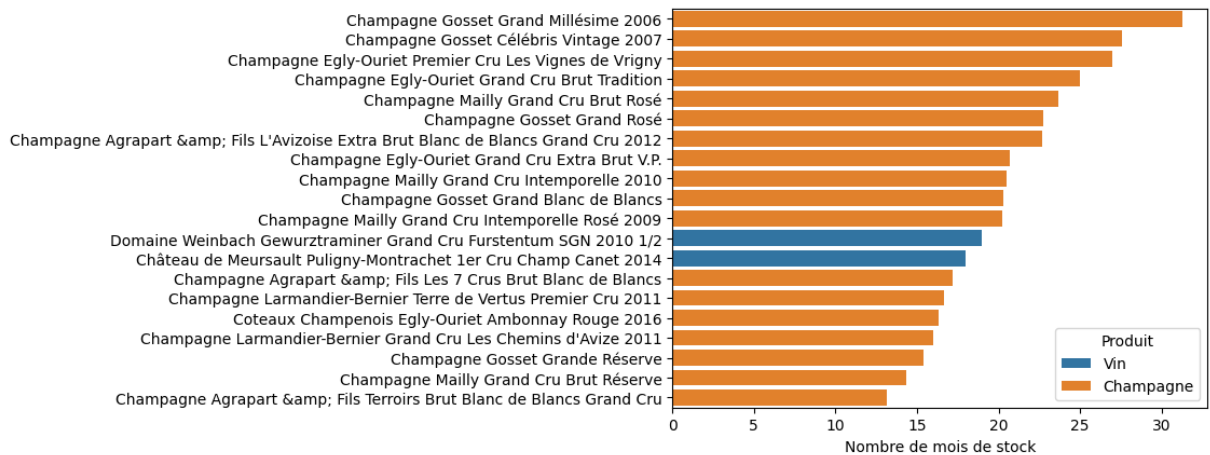
#Réinitialiser l'index du dataset par un reset_index
df_analyse_stock.reset_index(drop=True, inplace=True)

#Afficher les 20 premiers articles en quantité
df_analyse_stock.head(20)
```

product_id	price	stock_quantity	purchase_price	id_web	total_
------------	-------	----------------	----------------	--------	--------

0	4142	53.0	125	32.15	11641
1	6126	135.0	138	80.33	14923
2	4356	51.6	81	31.00	12585
3	4348	59.0	125	34.76	12586
4	4148	37.5	71	21.88	1364
5	4144	49.0	91	27.73	1662
6	5025	112.0	136	68.60	13914
7	4350	79.5	145	47.30	12588

```
#Graphique en barre du flop 20 des produits qui ont le plus de mois
sns.barplot(data=df_analyse_stock.head(20), x='month_stock', y='pos
plt.xlabel('Nombre de mois de stock')
plt.ylabel('')
plt.legend(title='Produit')
plt.show()
```



```
#####
```

```
# Valorisation des stocks en euros #
```

```
#####
```

```
#Création de la colonne Valorisation des stocks en euros
```

```
df_analyse_stock['stock_value'] = df_analyse_stock['stock_quantity'
```

```
#Calculer la somme de la colonne "Valorisation_stock"
```

```
print('Valeur du stock en octobre : {} €'.format(round(df_analyse_s
```

```
Valeur du stock en octobre : 284198.98 €
```

```
#####
# Valorisation du nombre de produits en stock #
#####

#Calculer la somme de la colonne stock quantity
print('Nombre de produits en stock en octobre : {} unités'.format(r
```

Nombre de produits en stock en octobre : 17461 unités

Etape 5.4 - Analyse du taux de marge

```
#####
# Analyse du taux de marge #
#####

df_analyse_marge=df_merge_final.copy()

#Création de la colonne Prix HT
df_analyse_marge['price_ht'] = df_analyse_marge['price'] / 1.2

#Création de la colonne Taux de marge
df_analyse_marge['taux_marge'] = (df_analyse_marge['price_ht'] - df

#Afficher le taux de marge minimum
taux_marge_min = df_analyse_marge['taux_marge'].min()
prix_vente_min = df_analyse_marge.loc[df_analyse_marge['taux_marge']
print('Le taux de marge minimum est de {}% pour un prix de vente de

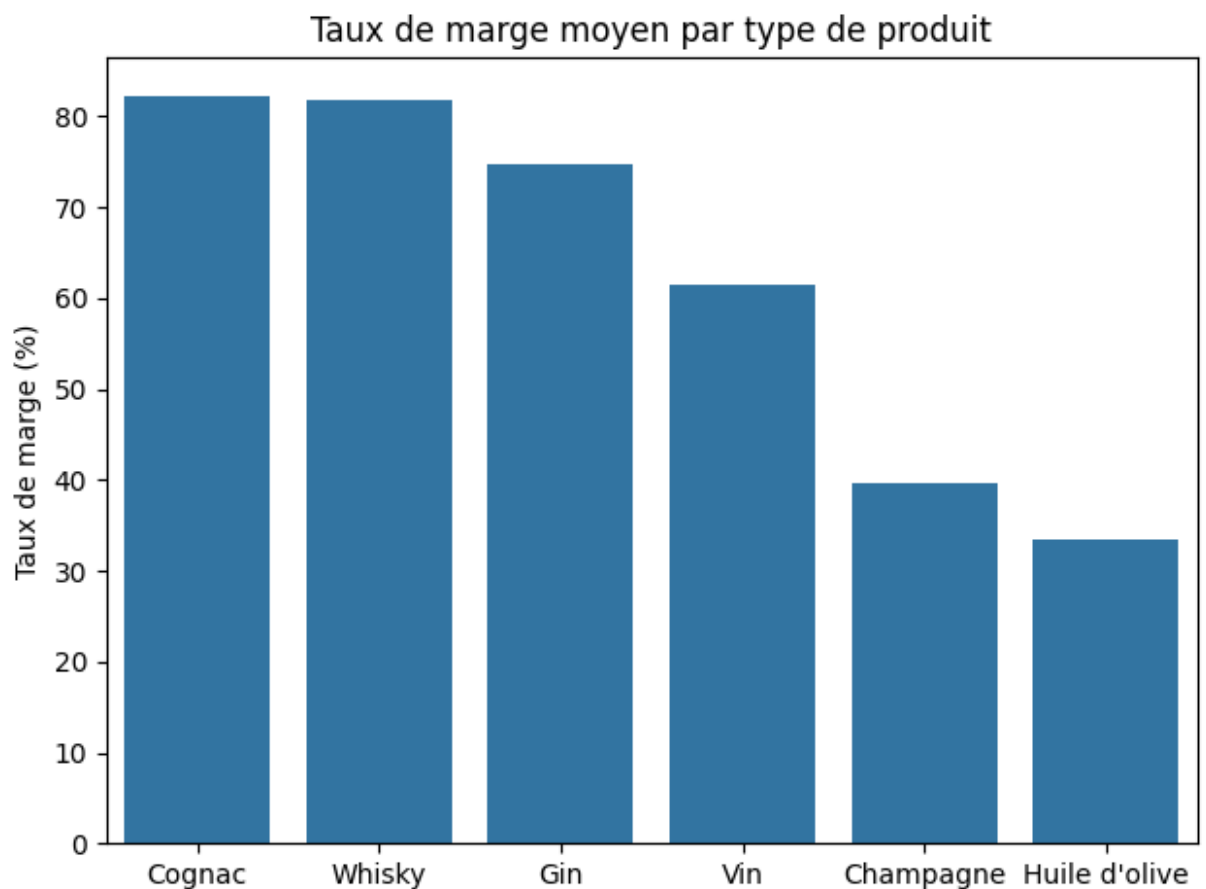
#Afficher le taux de marge maximum
taux_marge_max = df_analyse_marge['taux_marge'].max()
prix_vente_max = df_analyse_marge.loc[df_analyse_marge['taux_marge']
print('Le taux de marge maximum est de {}% pour un prix de vente de

#Afficher le taux de marge moyen
taux_marge_moyen = df_analyse_marge['taux_marge'].mean()
print('Le taux de marge moyen est de {}'.format(round(taux_marge_r
```

Le taux de marge minimum est de 29.5% pour un prix de vente de 13.1€
 Le taux de marge maximum est de 91.41% pour un prix de vente de 62.5€
 Le taux de marge moyen est de 61.25%

```
#Création d'un dataframe avec le taux de marge moyen par type de pr
df_analyse_marge_produit = df_analyse_marge.groupby('product_type')
df_analyse_marge_produit.sort_values(by='taux_marge', ascending=False)

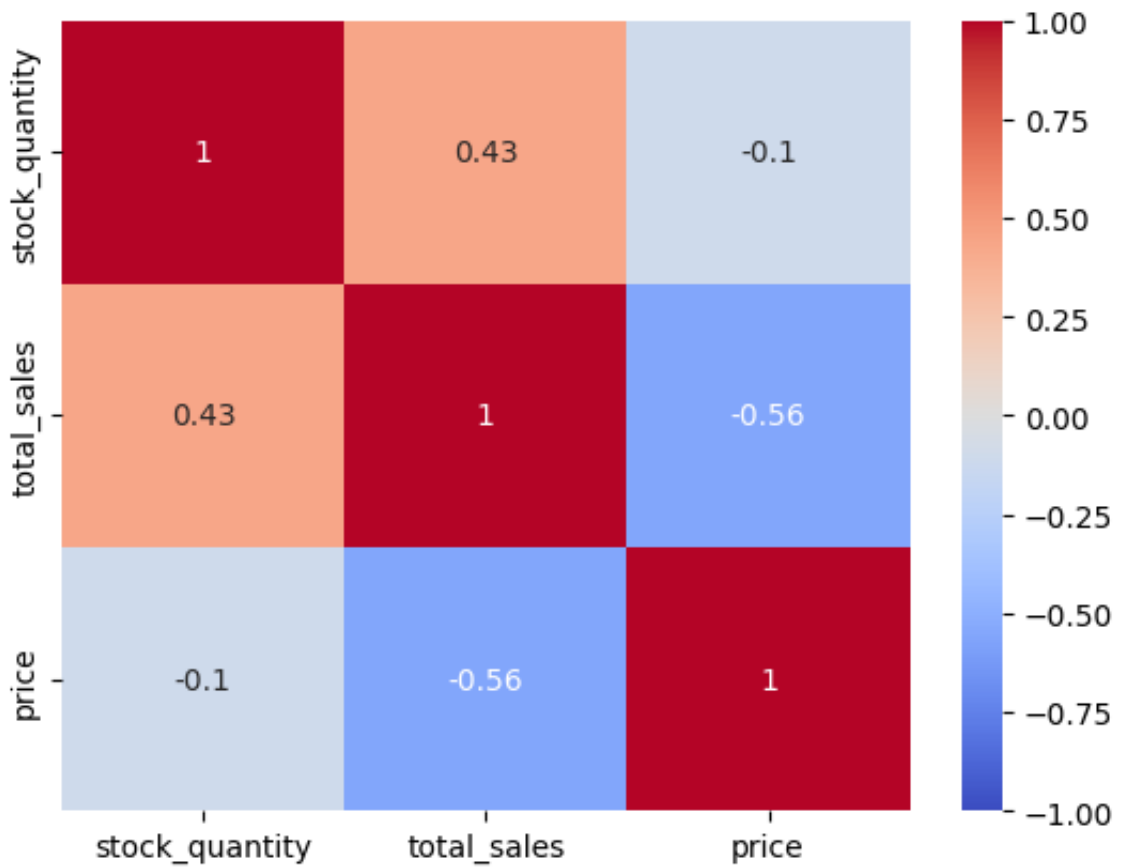
#Affichage dans un graphique du taux de marge par type de produit
sns.barplot(data=df_analyse_marge_produit, x='product_type', y='taux_marge')
plt.xlabel('')
plt.ylabel('Taux de marge (%)')
plt.title('Taux de marge moyen par type de produit')
plt.tight_layout()
plt.show()
```



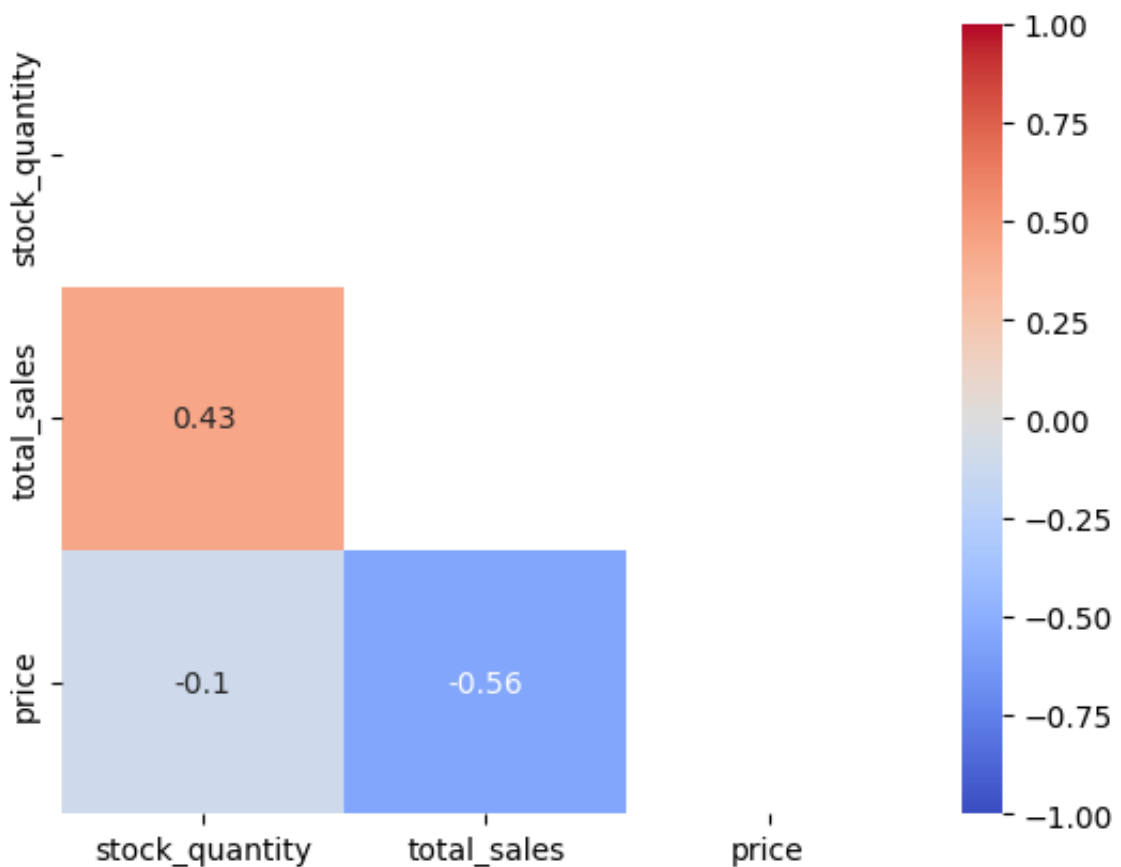
Etape 5.5 - Analyse des corrélations entre les variables stock, sales et price

```
#####  
# Analyse des corrélations #  
#####
```

```
#Création d'une heatmap de corrélation avec les variables stock, sa  
sns.heatmap(df_merge_final[['stock_quantity', 'total_sales', 'price  
plt.show()
```



```
#On peut également créer un mask pour n'afficher qu'une demi heatmap
mask = np.triu(np.ones_like(df_merge_final[['stock_quantity', 'total_sales', 'price'])))
sns.heatmap(df_merge_final[['stock_quantity', 'total_sales', 'price']], mask=mask)
plt.show()
```



Etape 5.6 - Mise à disposition de la nouvelle table sur un fichier Excel

```
#Selection des colonnes pertinentes
df_ca=df_analyse_ca[['product_id', 'ca', 'part_ca_total', 'priority']]
df_quantity=df_analyse_quantity[['product_id', 'part_quantity_total']]
df_stock=df_analyse_stock[['product_id', 'month_stock', 'stock_value']]
df_marge=df_analyse_marge[['product_id', 'price_ht', 'taux_marge']]
```

```
#Joindre les colonnes pertinentes dans une ultime dataframe
df_final=pd.merge(df_merge_final, df_ca, on='product_id', how='left')
df_final=pd.merge(df_final, df_quantity, on='product_id', how='left')
df_final=pd.merge(df_final, df_stock, on='product_id', how='left')
df_final=pd.merge(df_final, df_marge, on='product_id', how='left')
df_final.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 745 entries, 0 to 744
Data columns (total 17 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   product_id                            745 non-null    int64
1   price                                 745 non-null    float64
2   stock_quantity                        745 non-null    int64
3   purchase_price                        745 non-null    float64
4   id_web                                686 non-null    object
5   total_sales                           684 non-null    float64
6   product_type                          684 non-null    object
7   post_title                            684 non-null    object
8   ca                                     684 non-null    float64
9   part_ca_total                         684 non-null    float64
10  priority_ca                           745 non-null    object
11  part_quantity_total                   684 non-null    float64
12  priority_quantity                     745 non-null    object
13  month_stock                           684 non-null    float64
14  stock_value                           745 non-null    float64
15  price_ht                              745 non-null    float64
16  taux_marge                            745 non-null    float64
dtypes: float64(10), int64(2), object(5)
memory usage: 99.1+ KB
```

```
#Afficher les premières lignes de ce nouveau dataframe
df_final.head(5)
```

	product_id	price	stock_quantity	purchase_price	id_web	total_s
0	3847	24.2	16	12.88	15298	
1	3849	34.3	10	17.54	15296	

```
#Mettre le dataset df_merge sur un fichier Excel
df_final.to_excel('drive/MyDrive/Colab Notebooks/df_final.xlsx', in
```

3	4041	32.7	12	17.57	14975
4	4042	31.2	15	15.48	16042